HANZO FHE INFERENCE

Fully Homomorphic Encryption for Confidential AI Inference

Zach Kelling

Hanzo AI

March 2026

Abstract

We present Hanzo FHE Inference, a system for running machine learning models on encrypted data using Fully Homomorphic Encryption (FHE). The system enables privacy-preserving AI-as-a-service: clients encrypt their inputs locally, the Hanzo Cloud evaluates AI models on ciphertexts without decryption, and only the client can decrypt the result. We implement two FHE schemes optimized for different model architectures: (i) CKKS (Cheon–Kim–Kim–Song) for neural network inference, supporting approximate fixed-point arithmetic on encrypted vectors with SIMD batching, and (ii) TFHE (Torus FHE) for decision tree and gradient-boosted model evaluation, supporting exact Boolean and integer operations with programmable bootstrapping. We describe GPU acceleration of FHE operations using the Candle runtime [1], achieving 14× speedup over CPU-only evaluation on Number Theoretic Transform (NTT) kernels. Production deployment on Hanzo ACI [2] demonstrates encrypted inference on a 4-layer neural network in 1.8 seconds (CKKS) and encrypted XGBoost evaluation with 500 trees in 0.9 seconds (TFHE), enabling practical confidential AI for healthcare, finance, and regulated industries. The system integrates with the Lux TFHE infrastructure [6] for on-chain verification of encrypted computation proofs.

1 Introduction

AI-as-a-service requires clients to send sensitive data—medical records, financial transactions, legal documents—to cloud infrastructure for inference. This creates a fundamental privacy conflict: the service provider must see the data to process it, but the client must not reveal the data to the provider.

Existing approaches have significant limitations:

1. **Trusted execution environments (TEEs)**: Hardware enclaves (Intel SGX, AMD SEV) protect data in use but rely on hardware vendor trust and are vulnerable to side-channel attacks [10].

2. **Differential privacy**: Adds noise to outputs, protecting individual records but degrading model quality. Does not protect the input data itself during computation.

3. **Secure multi-party computation (MPC)**: Requires communication between parties during computation, introducing latency proportional to circuit depth.

4. **Federated learning**: Keeps data local during training but does not address the inference privacy problem.

FHE eliminates this conflict entirely: computation occurs on ciphertexts, the server never sees plaintext data, and correctness is mathematically guaranteed. The historical barrier—computational cost—has been reduced by four orders of magnitude through algorithmic improvements and GPU acceleration [9].

1.1 Contributions

1. A production FHE inference system supporting both CKKS and TFHE schemes, with automatic scheme selection based on model architecture (§3).

2. GPU-accelerated NTT and bootstrapping kernels implemented in the Candle framework, achieving 14× speedup over CPU baselines (§4).

3. Integration with Hanzo ACI for verifiable encrypted computation and Lux TFHE for on-chain proof verification (§7).

4. Production benchmarks on real workloads: encrypted medical image classification, encrypted credit scoring, and encrypted document embedding (§8).

2 FHE Background

Definition 1 (Fully Homomorphic Encryption)**.** *An FHE scheme* $\mathcal{E} = (\texttt{KeyGen}, \texttt{Enc}, \texttt{Dec}, \texttt{Eval})$ *satisfies: for any function* f *and plaintexts* $x_1, \ldots, x_k$*,*

$$\texttt{Dec}_{sk}(\texttt{Eval}_{ek}(f, \texttt{Enc}_{pk}(x_1), \ldots, \texttt{Enc}_{pk}(x_k))) = f(x_1, \ldots, x_k), \tag{1}$$

where $(pk, sk, ek) \leftarrow \texttt{KeyGen}(\lambda)$ *for security parameter* λ*.*

The computational cost of FHE depends on the depth of the arithmetic circuit being evaluated. Each homomorphic operation increases noise in the ciphertext; *bootstrapping* refreshes noise levels but is expensive.

2.1 CKKS for Neural Networks

CKKS [7] supports approximate arithmetic on encrypted real-valued vectors. A single ciphertext encodes $N/2$ complex values (or N real values) via SIMD packing, where N is the polynomial ring degree.

Definition 2 (CKKS Encoding)**.** *Given a plaintext vector* $\mathbf{z} \in \mathbb{C}^{N/2}$*, the encoding map* $\sigma^{-1} : \mathbb{C}^{N/2} \to \mathbb{Z}[X]/(X^N + 1)$ *produces a polynomial* $m(X)$ *such that* $\sigma(m) \approx \Delta \cdot \mathbf{z}$*, where* Δ *is a scaling factor controlling precision.*

CKKS supports:

- **Addition**: $\texttt{Enc}(a) + \texttt{Enc}(b) = \texttt{Enc}(a + b)$, consuming no multiplicative depth.
- **Multiplication**: $\texttt{Enc}(a) \cdot \texttt{Enc}(b) = \texttt{Enc}(a \cdot b)$, consuming one level of multiplicative depth.
- **Rotation**: $\texttt{Rot}(\texttt{Enc}(\mathbf{z}), k) = \texttt{Enc}(\text{rotate}(\mathbf{z}, k))$, enabling SIMD-parallel operations.

A neural network layer $\mathbf{y} = \phi(W\mathbf{x}+\mathbf{b})$ is evaluated homomorphically by encoding matrix-vector products as diagonal rotations and approximating activation functions with low-degree polynomials.

Proposition 1 (Polynomial Activation Approximation)**.** *The ReLU function* $\phi(x) = \max(0, x)$ *is approximated on* $[-B, B]$ *by a degree-d minimax polynomial* $\hat{\phi}_d(x)$ *with error:*

$$\|\phi - \hat{\phi}_d\|_\infty \leq \frac{2B}{\pi} \cdot \frac{1}{2^d}, \tag{2}$$

where $d = 7$ *suffices for* $<0.1\%$ *classification accuracy loss.*

2.2 TFHE for Decision Trees

TFHE [8] operates on encrypted bits and integers with exact arithmetic. Programmable bootstrapping allows evaluating arbitrary lookup tables on encrypted values in a single operation.

Definition 3 (Programmable Bootstrapping)**.** *Given a ciphertext c encrypting $m \in \mathbb{Z}_p$ and a function $f : \mathbb{Z}_p \to \mathbb{Z}_p$, programmable bootstrapping produces a fresh ciphertext encrypting $f(m)$ with reset noise:*

$$\mathit{PBS}(c, f) = \mathtt{Enc}(f(\mathtt{Dec}(c))), \tag{3}$$

without revealing m to the evaluator.

Decision tree evaluation reduces to a sequence of encrypted comparisons. Each tree node performs an encrypted comparison $\mathtt{Enc}(x_i) \overset{?}{<} t_j$ using TFHE's Boolean gates, and the result selects the left or right child.

Theorem 2 (Decision Tree Complexity)**.** *A decision tree of depth d with threshold values in $\mathbb{Z}_{2^b}$ requires d encrypted comparisons, each costing b programmable bootstrappings. Total cost: $O(d \cdot b)$ bootstrappings.*

For a gradient-boosted ensemble with T trees of depth d, the total cost is $O(T \cdot d \cdot b)$ bootstrappings. Crucially, all T trees can be evaluated in parallel.

3 Scheme Selection

The FHE inference system automatically selects the optimal scheme based on the model architecture:

Model Type	**Scheme**	**Rationale**
Dense neural network	CKKS	Approximate arithmetic, SIMD batching
Convolutional network	CKKS	Rotation-based convolution
Decision tree / forest	TFHE	Exact comparisons
Gradient-boosted model	TFHE	Parallel tree evaluation
Logistic regression	CKKS	Matrix-vector product
SVM (linear)	CKKS	Inner product + sign
SVM (kernel)	CKKS + TFHE	CKKS for kernel, TFHE for argmax

Table 1: Automatic scheme selection by model architecture.

For hybrid models (e.g., neural network with tree-based post-processing), the system uses *scheme switching*: CKKS ciphertexts are converted to TFHE ciphertexts at the boundary, incurring a one-time cost of $O(N \log N)$ operations.

4 GPU Acceleration

FHE's primary bottleneck is polynomial arithmetic over large rings ($N = 2^{14}$ to 2^{16}). We accelerate two critical operations on GPU using the Candle runtime [1].

4.1 Number Theoretic Transform

The Number Theoretic Transform (NTT) converts polynomials from coefficient to evaluation representation, enabling $O(N)$ pointwise multiplication instead of $O(N^2)$ schoolbook multiplication.

Definition 4 (NTT)**.** *For polynomial $a(X) = \sum_{i=0}^{N-1} a_i X^i$ over $\mathbb{Z}_q$, the NTT is:*

$$\hat{a}_j = \sum_{i=0}^{N-1} a_i \cdot \omega^{ij} \mod q, \quad j = 0, \ldots, N-1, \tag{4}$$

where ω is a primitive N-th root of unity modulo q.

Our GPU NTT kernel processes $N = 65536$ elements across 256 CUDA threads per block, with each thread performing butterfly operations on 256 elements. Shared memory stores twiddle factors to minimize global memory access.

Ring Degree N	**CPU (ms)**	**GPU (ms)**	**Speedup**
2^{14}	1.2	0.09	13.3×
2^{15}	2.6	0.18	14.4×
2^{16}	5.8	0.41	14.1×

Table 2: NTT performance: single-threaded CPU (AMD EPYC 9654) vs GPU (NVIDIA H100).

4.2 Bootstrapping Acceleration

TFHE bootstrapping dominates encrypted computation cost. Each bootstrapping operation performs a blind rotation (polynomial multiplication controlled by encrypted bits) followed by key switching.

On GPU, we parallelize across bootstrapping operations (independent across encrypted bits) and within each operation (NTT-based polynomial multiplication). A single H100 GPU executes 18,000 bootstrappings per second, compared to 1,200 on a 96-core CPU.

5 Encrypted Neural Network Inference

5.1 Layer-by-Layer Evaluation

A neural network with L layers is evaluated homomorphically as:

$$\mathbf{h}^{(l)} = \hat{\phi}_d\Big(W^{(l)} \cdot \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\Big), \quad l = 1, \ldots, L, \tag{5}$$

where $\mathbf{h}^{(0)} = \texttt{Enc}(\mathbf{x})$ is the encrypted input and $\hat{\phi}_d$ is the polynomial activation approximation.

Lemma 3 (Multiplicative Depth)**.** *An L-layer neural network with degree-d polynomial activations requires multiplicative depth $L \cdot (\lceil \log_2 d \rceil + 1)$. For $L = 4$ and $d = 7$: depth $= 4 \times (3+1) = 16$ levels.*

5.2 CKKS Parameter Selection

The ring degree N and ciphertext modulus Q are selected to provide 128-bit security while supporting the required multiplicative depth:

Depth	Ring N	$\log_2 Q$	Security (bits)
8	2^{14}	438	128
16	2^{15}	881	128
24	2^{16}	1761	128

Table 3: CKKS parameter selection for 128-bit security at various depths.

6 Encrypted Decision Tree Inference

6.1 Oblivious Tree Evaluation

To prevent information leakage from access patterns, we evaluate the entire tree obliviously: every path is evaluated, and the result is selected via encrypted multiplexing.

Algorithm 1 Oblivious Decision Tree Evaluation

Require: Encrypted input $\texttt{Enc}(\mathbf{x})$, tree $\mathcal{T}$ with nodes (i_v, t_v, l_v) for feature index, threshold, leaf value

Ensure: Encrypted prediction $\texttt{Enc}(y)$

for each internal node v **do**
  $b_v \leftarrow \texttt{Enc}(x_{i_v}) < t_v$ ▷ Encrypted comparison
end for
for each leaf ℓ with path $P_\ell = (v_1, d_1), \ldots, (v_k, d_k)$ **do**
  $s_\ell \leftarrow \prod_{(v,d)\in P_\ell} \begin{cases} b_v & \text{if } d = \texttt{left} \\ 1 - b_v & \text{if } d = \texttt{right} \end{cases}$
end for
$\texttt{Enc}(y) \leftarrow \sum_\ell s_\ell \cdot l_\ell$

6.2 Ensemble Parallelism

Gradient-boosted ensembles (XGBoost, LightGBM) consist of T independent trees. All trees are evaluated in parallel on GPU, with the encrypted results summed:

$$\texttt{Enc}(y) = \sum_{t=1}^{T} \texttt{Enc}(y_t). \tag{6}$$

This parallelism makes ensemble evaluation only marginally slower than single tree evaluation on GPU ($T = 500$ trees in 0.9s vs. single tree in 0.05s).

7 Integration with Hanzo and Lux Infrastructure

7.1 Hanzo ACI Verifiable Computation

FHE computations on Hanzo ACI nodes produce verifiable proofs:

1. The GPU node commits to the encrypted input, model weights, and encrypted output via a Merkle hash.

2. A SNARK proof attests that the committed encrypted output is the correct evaluation of the committed model on the committed encrypted input.

3. The proof is submitted to the ACI consensus layer for verification and on-chain attestation.

7.2 Lux TFHE Integration

The Lux blockchain provides native TFHE operations [6] for on-chain encrypted computation. Hanzo FHE Inference uses Lux TFHE for:

- **Key management**: FHE public keys are registered on-chain, enabling any ACI node to serve encrypted inference without key exchange.
- **Result verification**: Encrypted inference results are verified on-chain by re-executing a subset of operations.
- **Access control**: Decryption keys are managed via threshold secret sharing on-chain (§11), enabling multi-party access policies.

7.3 Hanzo KMS Integration

FHE key material is managed through Hanzo KMS [4]:

- Secret keys are stored encrypted at rest with AES-256-GCM envelope encryption.
- Key rotation follows the KMS per-environment rotation policy.
- All key access events are logged to the KMS audit trail.

8 Production Results

We benchmark three production workloads on H100 GPUs.

Workload	Scheme	Latency	Accuracy	vs. Plaintext
Medical image (4-layer CNN)	CKKS	1.8s	97.1%	−0.3%
Credit scoring (XGBoost 500)	TFHE	0.9s	99.8%	−0.0%
Document embedding (2-layer)	CKKS	0.6s	—	cosine > 0.998
Fraud detection (RF 200)	TFHE	0.4s	99.9%	−0.0%

Table 4: Encrypted inference benchmarks on NVIDIA H100 GPU. Accuracy compared to plaintext model evaluation.

CKKS-based neural network inference introduces minimal accuracy loss (<0.3%) from polynomial activation approximation. TFHE-based tree models preserve exact accuracy since TFHE performs exact integer arithmetic.

8.1 Throughput

With GPU acceleration, a single H100 sustains:

- CKKS: 35 encrypted neural network inferences per minute (4-layer, $N = 2^{15}$).
- TFHE: 67 encrypted XGBoost evaluations per minute (500 trees, depth 8, 16-bit features).

9 Security Analysis

Theorem 4 (IND-CPA Security)**.** *The Hanzo FHE Inference system achieves IND-CPA security under the Ring Learning with Errors (RLWE) assumption with parameters providing ≥128-bit classical security as estimated by the LWE Estimator [11].*

Remark 1. *FHE provides stronger guarantees than TEE-based approaches: security relies on mathematical hardness assumptions (RLWE), not on hardware trust. The RLWE problem is believed to be hard even for quantum adversaries, providing post-quantum security [5].*

10 Related Work

CrypTen [12] and **TenSEAL** [13] provide FHE wrappers for PyTorch tensors but lack GPU acceleration and production serving infrastructure.

Concrete ML [14] by Zama compiles scikit-learn models to FHE circuits using TFHE. Our system extends this approach with CKKS support for neural networks and GPU acceleration via Candle.

Jin [3] handles multimodal routing for Hanzo models; FHE inference integrates as an optional encrypted execution backend for privacy-sensitive requests.

11 Conclusion

Hanzo FHE Inference enables practical confidential AI-as-a-service by combining CKKS for neural networks, TFHE for decision trees, and GPU acceleration through the Candle runtime. Production deployments demonstrate sub-2-second encrypted inference with negligible accuracy loss, making FHE viable for regulated industries requiring data privacy guarantees stronger than any hardware-based approach. Integration with Hanzo ACI and Lux TFHE provides verifiable encrypted computation with on-chain attestation.

References

[1] Hanzo AI Research. Hanzo Candle: Rust-native ML framework for production inference. *Hanzo AI Research*, 2024.

[2] Hanzo AI Research. Hanzo ACI: AI Chain Infrastructure for decentralized compute markets. *Hanzo AI Research*, December 2024.

[3] Hanzo AI Research. Jin: Unified multimodal AI architecture. *Hanzo AI Research*, 2025.

[4] Hanzo AI Research. Hanzo KMS: Secrets management with per-environment rotation and audit trail. *Hanzo AI Research*, September 2021.

[5] Hanzo AI Research. Hanzo post-quantum cryptography: Migration path for AI infrastructure. *Hanzo AI Research*, April 2026.

[6] Lux Partners. Lux TFHE: On-chain fully homomorphic encryption. *Lux Network Technical Report*, 2024.

[7] J. H. Cheon, A. Kim, M. Kim, and Y. Song. Homomorphic encryption for arithmetic of approximate numbers. In *Proc. ASIACRYPT*, 2017.

[8] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène. TFHE: Fast fully homomorphic encryption over the torus. *J. Cryptology*, 33(1):34–91, 2020.

[9] C. Mouchet, J. Bossuat, J.-P. Bossuat, and J.-P. Hubaux. A survey of FHE for machine learning. *ACM Computing Surveys*, 2024.

[10] J. Van Bulck, M. Minkin, O. Weisse, D. Genkin, B. Gras, F. Piessens, M. Silberstein, T. F. Wenisch, Y. Yarom, and R. Strackx. Foreshadow: Extracting the keys to the Intel SGX kingdom. In *Proc. USENIX Security*, 2018.

[11] M. Albrecht, R. Player, and S. Scott. On the concrete hardness of Learning with Errors. *J. Mathematical Cryptology*, 9(3):169–203, 2015.

[12] B. Knott, S. Venkataraman, A. Hannun, S. Sengupta, M. Ibrahim, and L. van der Maaten. CrypTen: Secure multi-party computation meets machine learning. In *Proc. NeurIPS*, 2021.

[13] A. Benaissa, B. Retiat, B. Cebere, and A. E. Belfedhal. TenSEAL: A library for encrypted tensor operations using homomorphic encryption. In *ICLR Workshop on DPML*, 2021.

[14] Zama. Concrete ML: Privacy-preserving machine learning using FHE. *Zama Documentation*, 2023.